Sistem Mimarisi & Müşteri Kılavuzu

Nginx API Gateway & Saf Python Servisleri

DOKTOR COMPOSE ORTAMINDA MİKROSERVİS YÖNLENDİRMESİ

Bu belge, harici üçüncü parti web çatılarına (FastAPI, Flask vb.) ihtiyaç duymadan, yalnızca Python'ın yerleşik `http.server` modülü ve **Nginx Reverse Proxy** mimarisi kullanılarak oluşturulmuş mikroservis yapısının kurulum ve çalıştırma detaylarını içermektedir.

1. Proje Yapısı ve Mimari Bileşenler

api_one Servisi

Port 8000 üzerinde çalışan ve `/login` rotasına istek geldiğinde HTML formatında kullanıcı giriş arayüzü döndüren bağımsız Python konteyneri.

api_two Servisi

Port 8000 üzerinde çalışan ve `/logout` rotasına istek geldiğinde JSON formatında durum mesajı döndüren ikinci Python konteyneri.

Nginx Gateway

Dış dünyadan 80 portu üzerinden gelen tüm HTTP isteklerini karşılayıp iç ağdaki doğru mikroservise şeffaf şekilde proxy eden ana yönlendirici.

Docker Compose

Tüm servislerin bağımlılık sırasına göre izolasyonlu bir sanal ağ (bridge) üzerinde otomatik derlenip ayağa kaldırılmasını sağlayan orkestrasyon.

2. Klasör Ağacı Düzeni

```
nginx-api-gateway/
├── api_one/
│   ├── app.py
│   └── Dockerfile
├── api_two/
│   ├── app.py
│   └── Dockerfile
├── nginx/
│   └── nginx.conf
└── docker-compose.yml
```

Sıfır Bağımlılık (Zero-Dependency) Yaklaşımı

Bu çözümde standart Python standart kütüphaneleri tercih edildiği için ekstra paket yüklemesi (pip install) gerekmez. Konteyner imajları hafif tutulur ve güvenlik açığı riski en aza indirilir.

3. API Servislerinin Kod Yapısı

Her iki servis de Python'ın `HTTPServer` ve `BaseHTTPRequestHandler` sınıfları kullanılarak geliştirilmiştir.

api_one/app.py (Giriş Servisi)

```
from http.server import HTTPServer, BaseHTTPRequestHandler

HTML_CONTENT = """
<html lang="tr"><head><meta charset="UTF-8"><title>Giriş Yap</title>
<style>
  body { font-family: Arial, sans-serif; display: flex; justify-content: center;
         align-items: center; height: 100vh; margin: 0; background: #f4f4f9; }
  .card { background: white; padding: 2rem; border-radius: 8px; width: 300px; }
  input, button { width: 100%; padding: 8px; margin: 8px 0; box-sizing: border-box; }
  button { background: #007bff; color: white; border: none; cursor: pointer; }
</style></head>
<body><div class="card"><h2>Giriş Yap</h2><form>
  <input type="text" placeholder="Kullanıcı Adı" required>
  <input type="password" placeholder="Şifre" required>
  <button type="submit">Giriş</button>
</form></div></body></html>"""

class SimpleHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/login":
            self.send_response(200)
            self.send_header("Content-type", "text/html; charset=utf-8")
            self.end_headers()
            self.wfile.write(HTML_CONTENT.encode("utf-8"))
        else:
            self.send_response(404)
            self.end_headers()

if __name__ == "__main__":
    server = HTTPServer(("0.0.0.0", 8000), SimpleHandler)
    server.serve_forever()
```

api_two/app.py (Çıkış Servisi)

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import json

class SimpleHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/logout":
            self.send_response(200)
            self.send_header("Content-type", "application/json; charset=utf-8")
            self.end_headers()
            response = {"message": "Çıkış yapılmıştır."}
            self.wfile.write(json.dumps(response, ensure_ascii=False).encode("utf-8"))
        else:
            self.send_response(404)
            self.end_headers()

if __name__ == "__main__":
    server = HTTPServer(("0.0.0.0", 8000), SimpleHandler)
    server.serve_forever()
```

4. Nginx ve Docker Yapılandırması

nginx/nginx.conf

```
events { worker_connections 1024; }

http {
    upstream api_one_service { server api_one:8000; }
    upstream api_two_service { server api_two:8000; }

    server {
        listen 80;
        server_name localhost;

        location /login {
            proxy_pass http://api_one_service/login;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /logout {
            proxy_pass http://api_two_service/logout;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

docker-compose.yml

```
version: '3.8'

services:
  api_one:
    build: context: ./api_one
    container_name: api_one_container
    restart: always

  api_two:
    build: context: ./api_two
    container_name: api_two_container
    restart: always

  nginx:
    image: nginx:latest
    container_name: nginx_gateway
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - api_one
      - api_two
    restart: always
```

5. Çalıştırma ve Test Adımları

Sistemi Ayağa Kaldırma

Terminal üzerinden proje kök dizininde `docker compose up --build` komutunu çalıştırın.

- **Giriş Testi:** `http://localhost/login` (HTML Sayfası)
- **Çıkış Testi:** `http://localhost/logout` (JSON Yanıtı)